

Optimizations

Software Performance Optimization

- Code optimization
- Loop optimization

Code optimization

To produce an efficient code

-
- Two issues in code optimization is
 1. Semantic equivalence of the source program must not be changed
 2. It must be achieved without changing the algorithm of the program

Code optimization techniques

- Compile Time evaluation
 - Constant folding and Constant Propagation
- Common Subexpression elimination
- Code movement
- Strength reduction
- Dead Code elimination

Compile Time evaluation

Constant folding and Constant Propagation

- We optimize the code at the compile time – two techniques -
 - constant folding and constant propagation
 - **Constant folding** – technique of evaluating an expression whose operands are known to be constants at compile time itself

① Compile Time Evaluation

i) Constant Folding:

It refers to a technique of evaluating the expressions whose operands are known to be constant at compile time itself.

Example:- $\text{length} = (2217) * d$

Compile Time evaluation

Constant folding and Constant Propagation

- In Constant Propagation if a variable is assigned a constant value then subsequent use of that variable can be replaced by a constant as long as no intervening assignment has changed the value of the variable
- So values of variables are directly substituted at the place of reference or usage and executed at compile time itself

Example:

$$\beta l = 3.14$$

$$r = 5$$

$$\text{Area} = \beta l * r * r$$

Here, the value of βl is replaced by 3.14 and r by 5, then computation of $3.14 * r * r$ is done during compilation.

Common Subexpression elimination

- It is an expression appearing repeatedly in the code which is computed previously.
- This technique replaces the redundant expression

Example:-

$T_1 = 4 * i$
 $T_2 = a[T_1]$
 $T_3 = 4 * j$
 $T_4 = 4 * i$
 $T_5 = n$
 $T_6 = b[T_4] + T_5$

Before Optimization

$T_1 = 4 * i$
 $T_2 = a[T_1]$
 $T_3 = 4 * j$
 $T_5 = n$
 $T_6 = b[T_4] + T_5$

After Optimization

Common Subexpression elimination

- Above is the 3 address code
- So $4 * i$ is not executed second time for T_4 and value of T_1 is taken for T_4
- So no redundant expression in the optimized code

Example:-

$T_1 = 4 * i$		$T_1 = 4 * i$
$T_2 = a[T_1]$		$T_2 = a[T_1]$
$T_3 = 4 * j$		$T_3 = 4 * j$
$T_4 = 4 * i$		$T_5 = n$
$T_5 = n$		$T_6 = b[T_4] + T_5$
$T_6 = b[T_4] + T_5$		

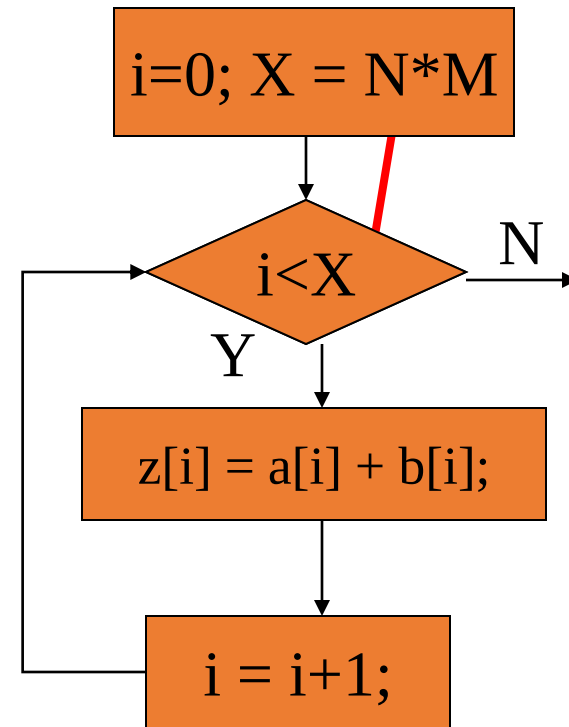
Before Optimization After Optimization

Code movement or code motion

- It is a technique of moving a block of code outside a loop if it does not have any difference if it is executed outside or inside the loop
- It reduces the evaluation frequency of expression

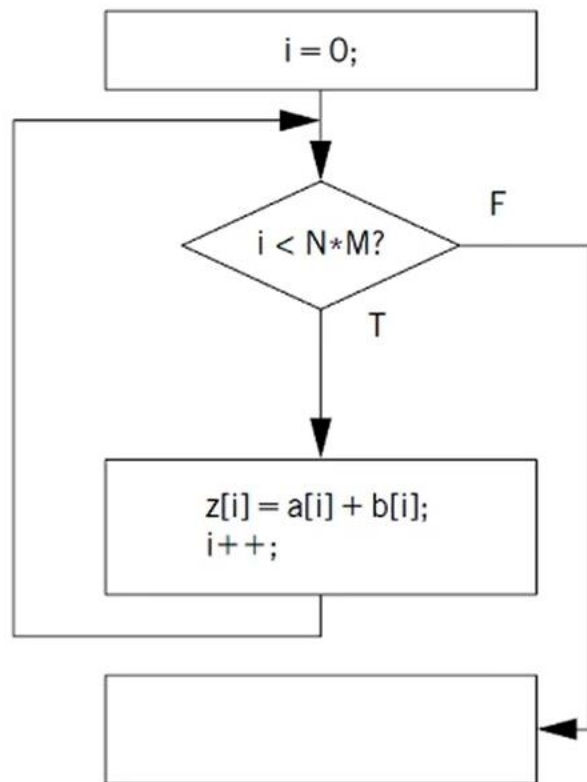
Code movement or code motion

```
for (i=0; i<N*M; i++)  
    z[i] = a[i] + b[i];
```

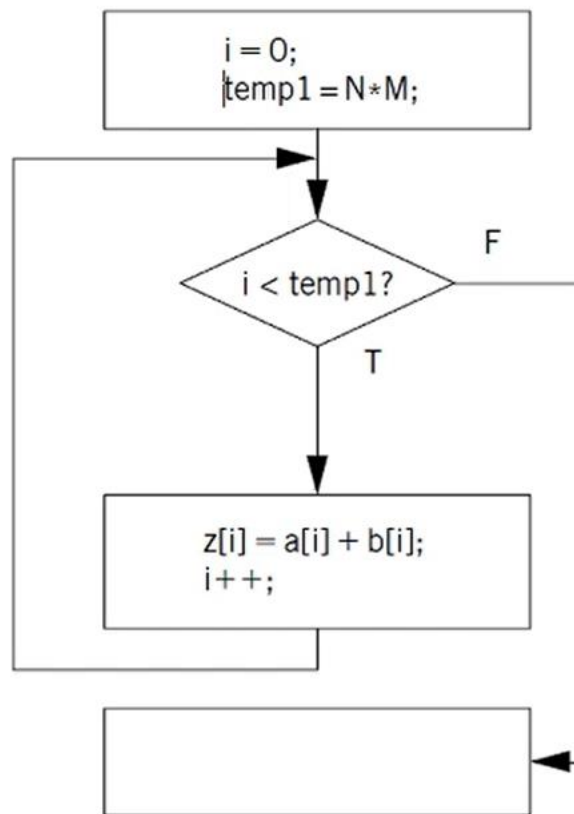


Consider the following loop:

```
for (i = 0; i < N*M; i++)  
{  
    z[i] = a[i] + b[i];  
}
```



The loop bound computation
→ result never changes.
→ We can avoid $N * M - 1$ unnecessary executions by moving it before the loop



Dead code elimination

- Code that will never be executed can be safely removed from the program

④ Dead Code Elimination:-

Dead Code Elimination includes eliminating those code statements which are either never executed or unreachable or if executed their output is never used.

Example:-

```
i = 0
if (i == 1)
{
    a = x + 5;
}
```

Before Optimization

```
i = 0
```

After Optimization

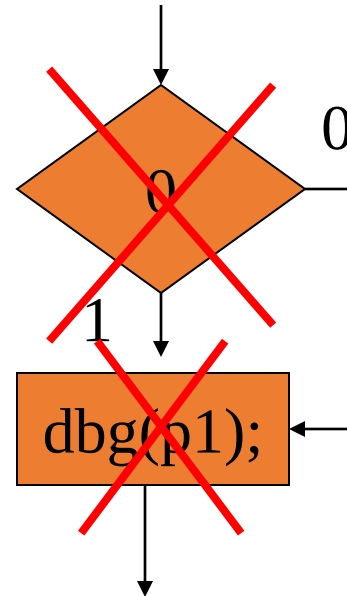
Dead code elimination

- Dead code:
- Code that will never be executed can be safely removed from the program.

```
#define DEBUG 0
```

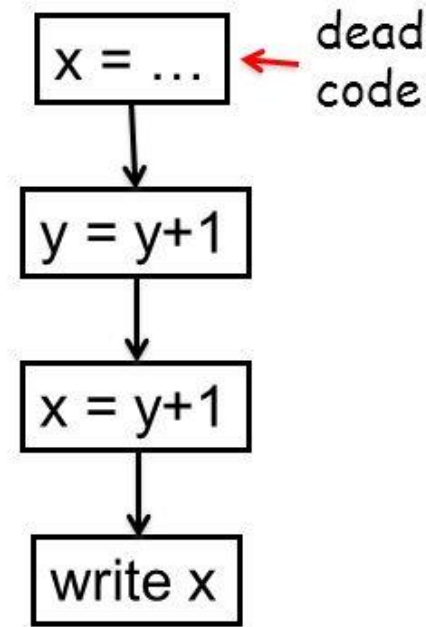
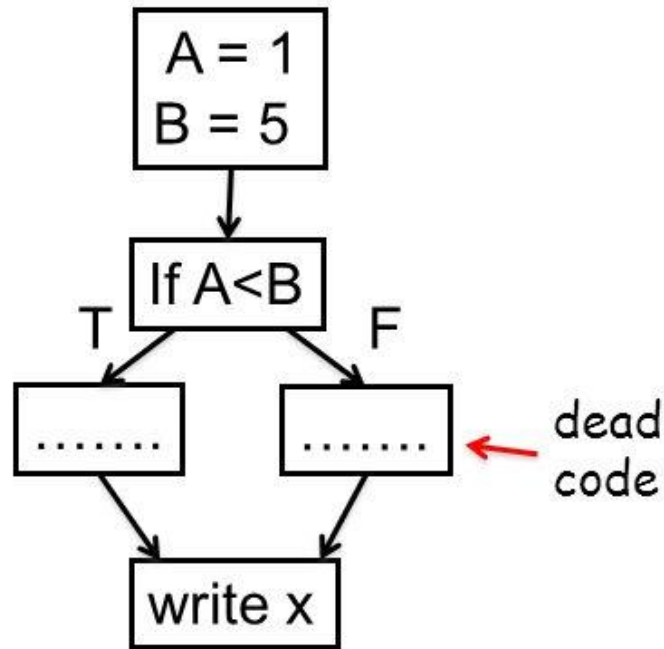
```
if (DEBUG) dbg(p1);
```

- Can be eliminated by analysis of control flow, constant folding.



Dead Code Elimination

Dead Code is code that is either never executed or, if it is executed, its result is never used by the program. Therefore dead code can be eliminated from the program.



Strength Reduction

- Replace an expression that are expensive with cheaper and simple ones
- Replace an expression having more priority with less priority expression

⑤ Strength Reduction:

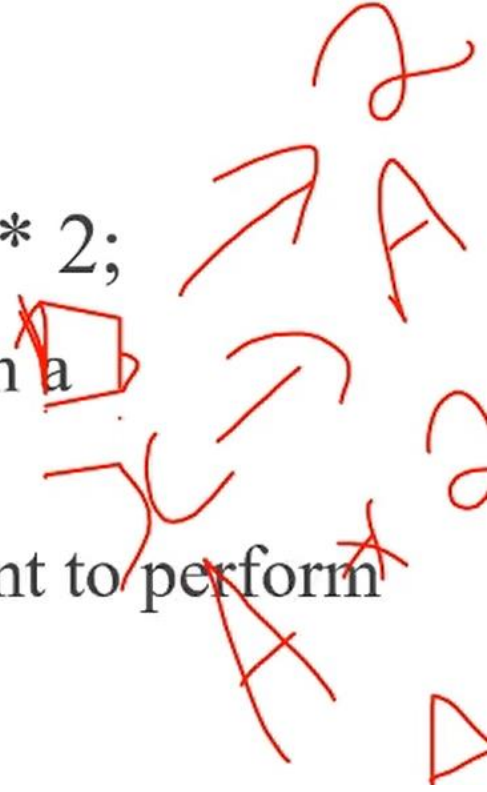
It is the replacement of expressions that are expensive with cheaper and simple ones.

Example:

$B = A * 2$	$B = A + A$
<u>Before Optimization</u>	<u>After Optimization</u>

Strength reduction

- ▶ Helps us reduce the cost of a loop iteration.
- ▶ Consider the following assignment: $y = x * 2;$
 - ▶ In integer arithmetic, we can use a left shift rather than a multiplication by 2
 - ▶ If the shift is faster than the multiply, we probably want to perform the substitution.



- Induction Variables and Strength Reduction

- A variable X is called an induction variable of a loop L if every time the variable X changed value, it is incremented or decremented by some constant
- When there are 2 or more induction variables in a loop, it may be possible to get rid of all but one
- It is also frequently possible to perform strength reduction on induction variables
 - the strength of an instruction corresponds to its execution cost
- **Benefit:** fewer and less expensive operations

```
t4 = 0
label_xxx
j = j + 1
t4 = 4 * j
t5 = a[t4]
if (t5 > v) goto label_xxx
```

Before

```
t4 = 0
label_xxx
t4 += 4
t5 = a[t4]
if (t5 > v) goto label_xxx
```

After

Loop optimizations

- Goals

- reduce loop overhead;
- increase opportunities for pipelining;
- improve memory system performance.

Loop optimizations techniques

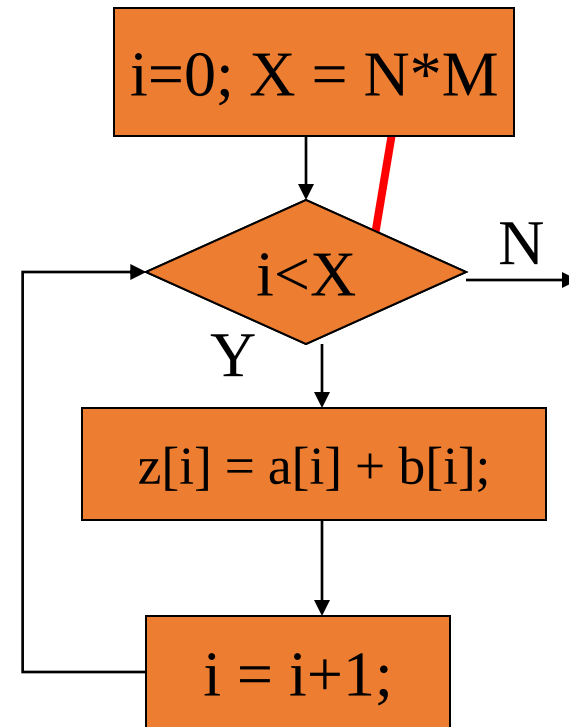
- **Code motion technique or code movement**
- **Induction variable**
- **Loop invariant method**
- **Loop unrolling**
- **Loop fusion and distribution or Loop jamming**
- **Loop tiling**

Code movement or code motion

- It is a technique of moving a block of code outside a loop if it does not have any difference if it is executed outside or inside the loop
- It reduces the evaluation frequency of expression

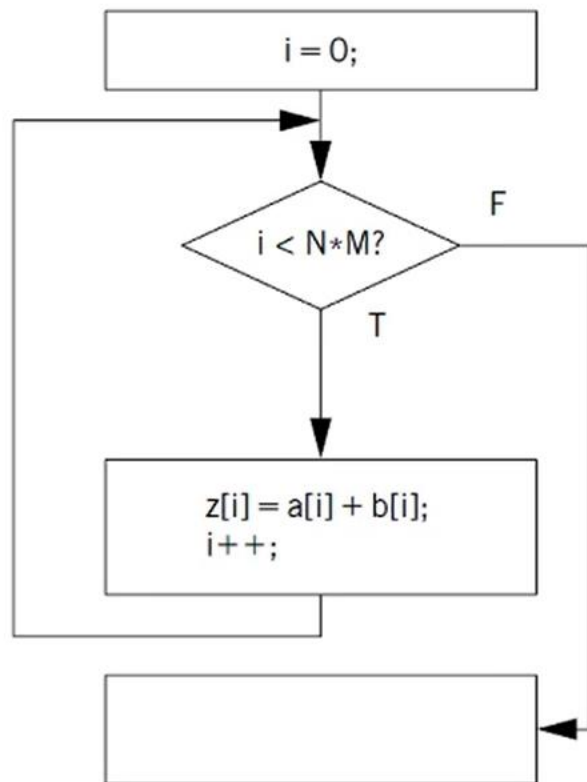
Code movement or code motion

```
for (i=0; i<N*M; i++)  
    z[i] = a[i] + b[i];
```

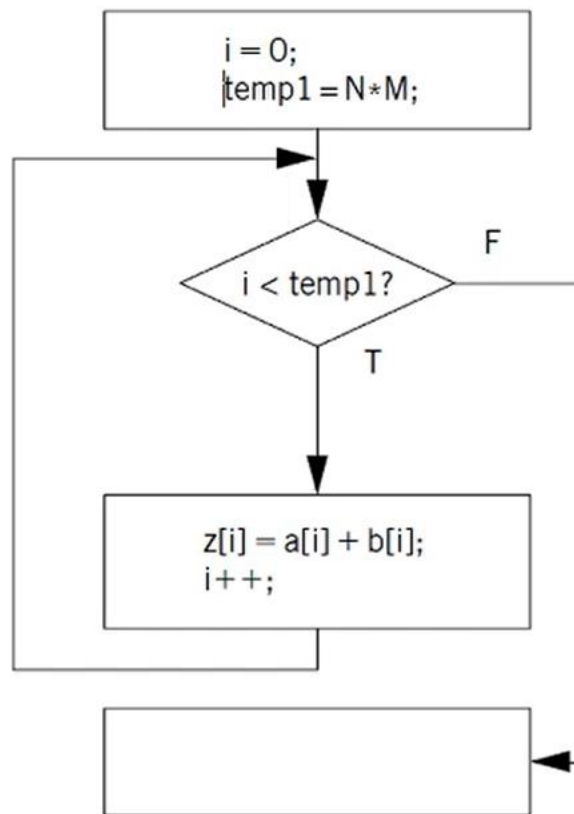


Consider the following loop:

```
for (i = 0; i < N*M; i++)  
{  
    z[i] = a[i] + b[i];  
}
```



The loop bound computation
→ result never changes.
→ We can avoid $N * M - 1$ unnecessary executions by moving it before the loop



Loop invariant method

- Computation inside the loop is avoided and thereby computation overhead on compiler is avoided

```
for i=0 to 10
```

```
  K= i+a/b
```

```
  ---
```

```
  ----
```

```
end
```

```
t=a/b
```

```
For i=0 to 10
```

```
  K=i+t
```

```
  ----
```

```
  ----
```

```
end
```

Loop unrolling

- Reduces loop overhead, enables some other optimizations.

```
for (i=0; i<4; i++)
```

```
    a[i] = b[i] * c[i];
```



```
for (i=0; i<2; i++) {
```

```
    a[i*2] = b[i*2] * c[i*2];
```

```
    a[i*2+1] = b[i*2+1] * c[i*2+1];
```

```
}
```

Loop unrolling

```
for (i=1; i<=3; i++)  
{  
    p-f("hi");  
}  
    ↓↓  
p-f("hi")  
p-f("hi")  
p-f("hi")
```

```
for (i=1; i<=200; i++)  
{  
    calc();  
}  
    ↓↓  
for (i=1; i<=100; i++)  
{  
    calc();  
    calc();  
}
```


Loop fusion and distribution

- Fusion combines two loops into 1:

```
for (i=0; i<N; i++) a[i] = b[i] * 5;
```

```
for (j=0; j<N; j++) w[j] = c[j] * d[j];
```

```
↯ for (i=0; i<N; i++) {  
    a[i] = b[i] * 5; w[i] = c[i] * d[i];  
}
```

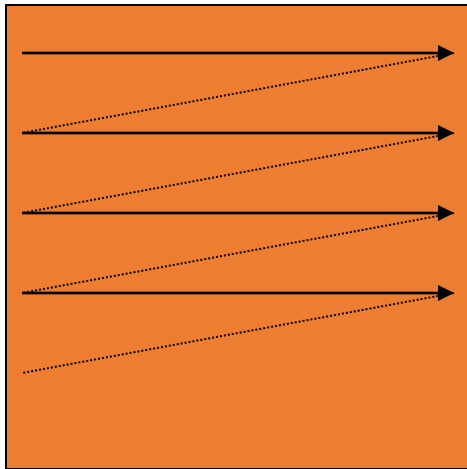
- Distribution breaks one loop into two.
- Changes optimizations within loop body.

Loop tiling

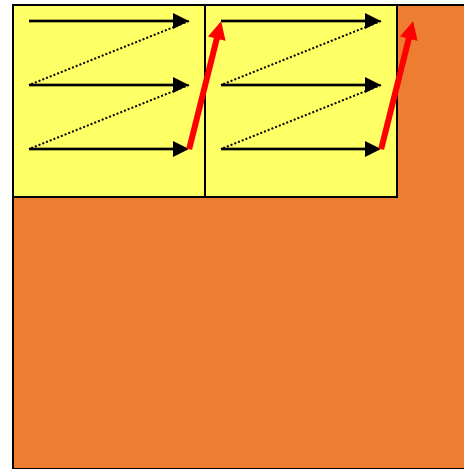
- Breaks one loop into a nest of loops.
- Changes order of accesses within array.
 - Changes cache behavior.

Loop tiling example

```
for (i=0; i<N; i++)  
  for (j=0; j<N; j++)  
    c[i] = a[i,j]*b[i];
```



```
for (i=0; i<N; i+=2)  
  for (j=0; j<N; j+=2)  
    for (ii=0; ii<min(i+2,n); ii++)  
      for (jj=0; jj<min(j+2,N); jj++)  
        c[ii] = a[ii,jj]*b[ii];
```

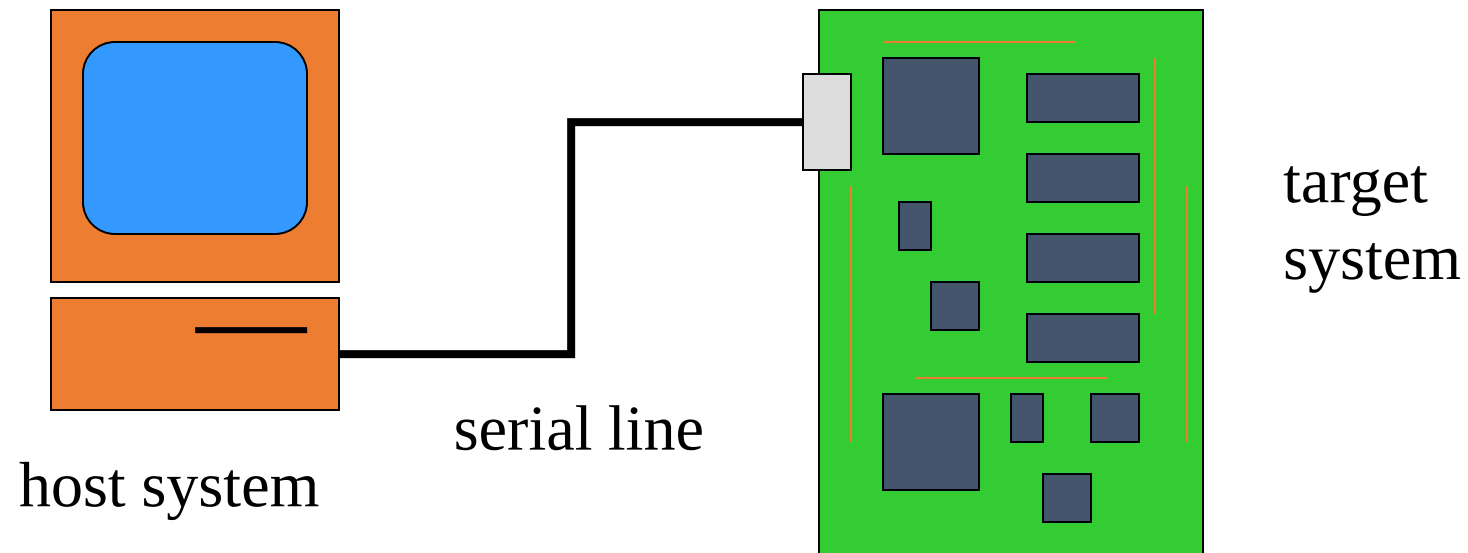


Debugging embedded systems

- Challenges:
 - target system may be hard to observe;
 - target may be hard to control;
 - may be hard to generate realistic inputs;
 - setup sequence may be complex.

Host/target design

- Use a host system to prepare software for target system:



Host-based tools

- Cross compiler:
 - compiles code on host for target system.
- Cross debugger:
 - displays target state, allows target system to be controlled.

Software debuggers

- A monitor program residing on the target provides basic debugger functions.
- Debugger should have a minimal footprint in memory.
- User program must be careful not to destroy debugger program, but , should be able to recover from some damage caused by user code.

Breakpoints

- A breakpoint allows the user to stop execution, examine system state, and change state.
- Replace the breakpointed instruction with a subroutine call to the monitor program.

ARM breakpoints

0x400 MUL r4,r6,r6

0x404 ADD r2,r2,r4

0x408 ADD r0,r0,#1

0x40c B loop

0x400 MUL r4,r6,r6

0x404 ADD r2,r2,r4

0x408 ADD r0,r0,#1

0x40c BL bkpnt

uninstrumented code

code with breakpoint

Breakpoint handler actions

- Save registers.
- Allow user to examine machine.
- Before returning, restore system state.
 - Safest way to execute the instruction is to replace it and execute in place.
 - Put another breakpoint after the replaced breakpoint to allow restoring the original breakpoint.

In-circuit emulators

- A microprocessor in-circuit emulator is a specially-instrumented microprocessor.
- Allows you to stop execution, examine CPU state, modify registers.

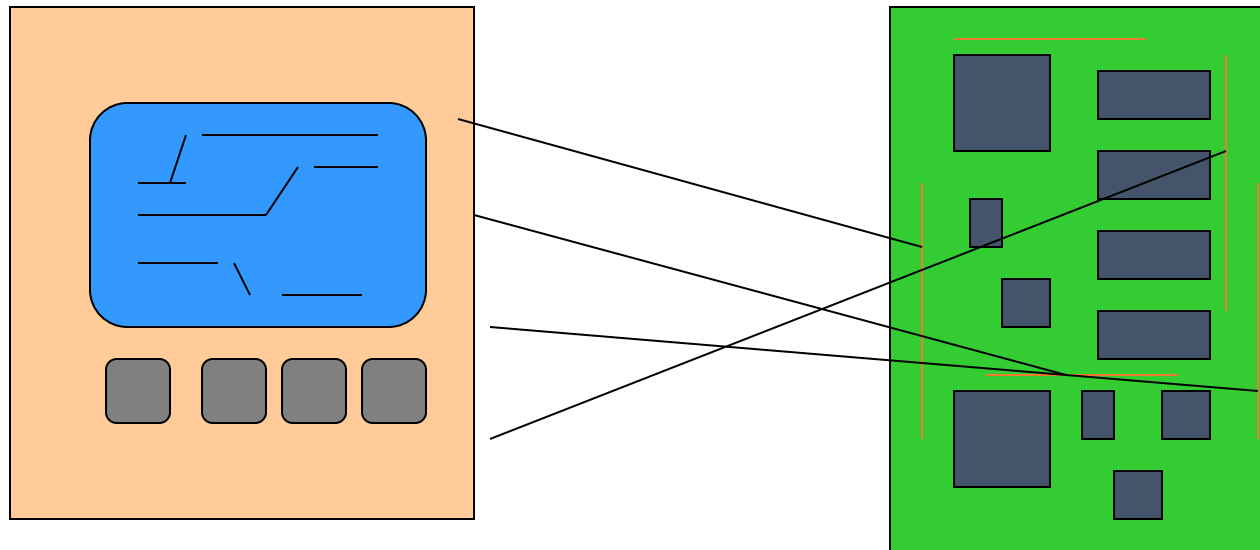
In-circuit emulators

- When software tools are insufficient to debug the system, hardware aids can be deployed to give a clear view of what is happening when the system is running.
- **The microprocessor-in-circuit emulator (ICE)** is a specialized hardware tool that can help debug software in a working embedded system

Logic analyzers

<https://www.youtube.com/watch?v=zIU9TryuHA>

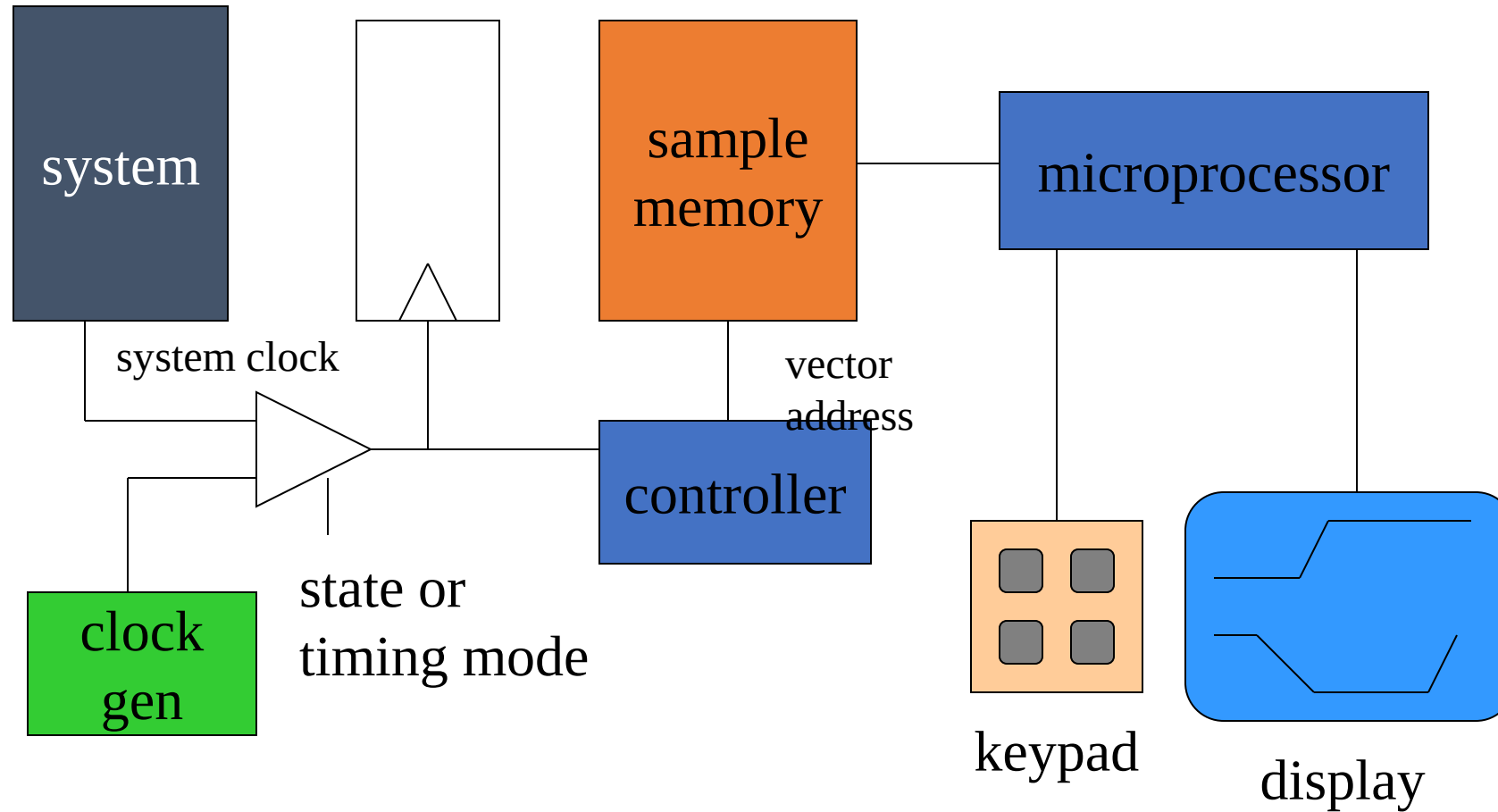
- A logic analyzer is an array of low-grade oscilloscopes:



Logic analyzer architecture

- **The** analyzer can sample many different signals simultaneously (tens to hundreds) but can display only 0,1,or changing values for each.
- All these logic analysis channels can be connected to the system to record the activity on many signals simultaneously.
- The logic analyzer records the values on the signals into an internal memory and then displays the results on a display once the memory is full or the run is aborted.
- The logic analyzer can capture thousands or even millions of samples of data on all of these channels,providing a much larger time window into the operation of the machine than is possible with a conventional oscilloscope.

Logic analyzer architecture



Logic analyzer architecture

- The system's data signals are sampled at a latch within the logic analyzer; the latch is controlled by either the system clock or the internal logic analyzer sampling clock, depending on whether the analyzer is being used in **state or timing mode**.
- Each sample is copied into a vector memory under the control of a state machine.
- The latch, timing circuitry, sample memory, and controller must be designed to run at high speed since several samples per system clock cycle may be required in timing mode

Logic analyzer architecture

- After the sampling is complete, an embedded microprocessor takes over to control the display of the data captured in the sample memory.
- Logic analyzers typically provide a number of formats for viewing data. One format is a timing diagram format.
- Many logic analyzers allow not only customized displays, such as giving names to signals, but also more advanced display options.
- For example, an inverse assembler can be used to turn vector values into microprocessor instructions.
- The logic analyzer does not provide access to the internal state of the components, but it does give a very good view of the externally visible signals. That information can be used for both functional and timing debugging.

Logic analyzer architecture

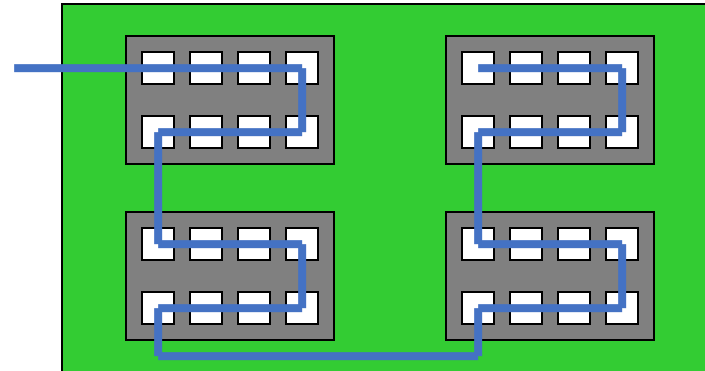
- A typical logic analyzer can acquire data in either of two modes that are typically called **state and timing modes**
- State and timing mode represent different ways of sampling the values.
- **Timing mode uses an internal clock that is fast enough to take several samples per clock period in a typical system.**
- **State mode, on the other hand, uses the system's own clock to control sampling, so it samples each signal only once per clock cycle.**

Advantages of Timing mode

- As a result, timing mode requires more memory to store a given number of system clock cycles.
- On the other hand, it provides greater resolution in the signal for detecting glitches.
- Timing mode is typically used for glitch-oriented debugging, while state mode is used for sequentially oriented problems.

Boundary scan

- Simplifies testing of multiple chips on a board.
 - Registers on pins can be configured as a scan chain.
 - Used for debuggers, in-circuit emulators.



How to exercise code

- Run on host system.
- Run on target system.
- Run in instruction-level simulator.
- Run on cycle-accurate simulator.
- Run in hardware/software co-simulation environment.

Debugging real-time code

- Bugs in drivers can cause non-deterministic behavior in the foreground problem.
- Bugs may be timing-dependent.